SmartDialer: Real-Time Dual-Agent Voice Pipeline, Sliding-Window Chunked ASR, and WebRTC Telephony

Full Technical Implementation and Knowledge Transfer Report

An Exhaustive Guide to Sliding-Window Audio Buffering, Faster-Whisper, NVIDIA Parakeet CTC, and LiveKit WebRTC Migration

SmartDialer Engineering Team
Waleed Tahir

AI Systems Architecture Group
`https://github.com/waleed-tahir/smartdailer`

October 10, 2026 — Version 1.0.0-PROD

Abstract

This document provides an exhaustive, production-grade technical implementation and knowledge transfer report for the **SmartDialer** platform. The platform is an autonomous, ultra-low-latency dual-agent voice system capable of orchestrating human-like dialogues between autonomous agents or between an automated dialer and a human telephone subscriber. We present the mathematical formulation and software engineering behind a 2.5-second streaming sliding window with a 0.5-second overlap ($T_w = 2.5\text{s}, T_o = 0.5\text{s}, T_s = 2.0\text{s}$) that completely eliminates boundary truncation artifacts while maintaining sub-second conversational responsiveness.

Furthermore, we document the comparative implementation of two distinct state-of-the-art Automatic Speech Recognition (ASR) backends: CTranslate2-accelerated Faster-Whisper running in FP16 precision on an NVIDIA GeForce RTX 3060 Laptop GPU, and the NVIDIA NeMo 3.0 Conformer-CTC Parakeet architecture (`nvidia/parakeet-ctc-0.6b`). We analyze speech synthesis implementations (direct Windows SAPI5 non-blocking COM streams and asynchronous Edge-TTS), multi-agent dialogue turn-taking state machines, transcript deduplication mechanics, and the end-to-end porting of this system to the LiveKit WebRTC and SIP telephony infrastructure. Comprehensive post-mortems of system-level issues—including Windows CUDA dynamic link library (DLL) binding, COM apartment thread synchronization, CP1252 terminal encoding anomalies, and cuDNN shared-library symbol discrepancies—are documented in detail to serve as an authoritative operational guide for incoming engineering personnel.

Contents

1 Executive Summary and Project Architecture **3**
- 1.1 Introduction and Problem Statement . . . 3
- 1.2 Core System Requirements . . . 3
- 1.3 High-Level Architectural Topology . . . 4

2 Mathematical Formulation of Sliding-Window Streaming ASR **5**
- 2.1 Audio Representation and Discretization . . . 5
- 2.2 Sliding Window Geometry . . . 5
- 2.3 Remainder Flushing and Boundary Completeness . . . 6

3 Comparative Architecture of ASR Backends **7**
- 3.1 Faster-Whisper: Architecture and CUDA FP16 Optimization . . . 7
  - 3.1.1 Model Topology . . . 7
  - 3.1.2 CTranslate2 Optimizations . . . 7
- 3.2 NVIDIA Parakeet: NeMo Conformer-CTC Architecture . . . 8
  - 3.2.1 Conformer Architecture . . . 8
  - 3.2.2 Connectionist Temporal Classification (CTC) . . . 8

4 Transcript Reconciliation and Boundary Deduplication **9**
- 4.1 The Acoustic Boundary Overlap Problem . . . 9
- 4.2 Token Normalization and Boundary Alignment . . . 9
  - 4.2.1 Token Normalization Function . . . 9
  - 4.2.2 Suffix-Prefix Longest Common Match . . . 10
  - 4.2.3 Subsequence and Truncated Syllable Recovery . . . 10

5 Speech Synthesis and Audio Output Architecture **12**
- 5.1 Real-Time Text-to-Speech Strategy . . . 12
- 5.2 Windows SAPI5 Non-Blocking COM Architecture . . . 12
  - 5.2.1 COM Apartment Threading and Stream Redirection . . . 12
- 5.3 Real-Time Physical Audio Output with SoundDevice . . . 13

6 Autonomous Dual-Agent Conversational Orchestration **14**
- 6.1 Turn-Taking State Machine . . . 14
- 6.2 Cognitive Brain Implementations . . . 14
  - 6.2.1 Deterministic Scenario Brain (`ScenarioBrain`) . . . 15
  - 6.2.2 Large Language Model Brain (`LLMBrain`) . . . 15

7 LiveKit WebRTC Integration and Telephony Architecture **16**
- 7.1 WebRTC Protocol Architecture and Audio Transport . . . 16
- 7.2 Custom LiveKit STT Adapter (`LiveKitChunkedSTT`) . . . 16
  - 7.2.1 Adapter Architecture . . . 16
- 7.3 LiveKit Audio Publishing (`LiveKitAudioPublisher`) . . . 17

7.4 SIP Telephony Integration 17

8 Engineering Post-Mortem and Technical Challenges Resolved 18
8.1 Issue 1: CTranslate2 Windows CUDA DLL Resolution Failure 18
8.1.1 Symptom 18
8.1.2 Root Cause 18
8.1.3 Resolution 18
8.2 Issue 2: SAPI5 COM Deadlocks During Multi-Turn Synthesis 19
8.2.1 Symptom 19
8.2.2 Root Cause 19
8.2.3 Resolution 19
8.3 Issue 3: Windows Console CP1252 Unicode Character Encoding 19
8.3.1 Symptom 19
8.3.2 Root Cause 19
8.3.3 Resolution 20
8.4 Issue 4: PyTorch and CTranslate2 cuDNN DLL Version Conflict 20
8.4.1 Symptom 20
8.4.2 Root Cause 20
8.4.3 Resolution 20
8.5 Issue 5: LiveKit RecognizeStream Lifecycle Deadlock 20
8.5.1 Symptom 20
8.5.2 Root Cause 20
8.5.3 Resolution 21

9 Developer Guide, Knowledge Transfer, and Operational Manual 22
9.1 Repository Structure 22
9.2 Environment Setup with UV 23
9.3 Execution Reference Manual 23
9.3.1 Standalone In-Memory Conversation 23
9.3.2 LiveKit WebRTC Integration 23
9.4 Operational Checklist for Incoming Engineers 23

Chapter 1

Executive Summary and Project Architecture

1.1 Introduction and Problem Statement

Real-time conversational voice systems represent one of the most challenging frontiers in modern applied artificial intelligence. Unlike text-based conversational interfaces where network latency of several seconds is tolerable, conversational voice agents operate within rigid psychoacoustic constraints. Human dialogue exhibits an average inter-turn latency of approximately 200 to 400 milliseconds. When an artificial conversational agent introduces latencies exceeding 700 to 1,000 milliseconds, conversational flow degrades, inducing hesitation, false barge-ins, and user frustration.

Traditional Automatic Speech Recognition (ASR) systems present an architectural dilemma:

1. **Full-Utterance Batch ASR**: The system employs Voice Activity Detection (VAD) to await the complete cessation of user speech (typically requiring 500–800 ms of trailing silence) before passing the accumulated audio buffer to an acoustic model (such as OpenAI Whisper). While transcription accuracy is maximized because the model observes the full acoustic and linguistic context, the resulting latency is catastrophic ($>$ 1.5–2.5 seconds post-speech).

2. **Instantaneous Frame-by-Frame Streaming ASR**: Acoustic models evaluate tiny audio slices (e.g., 20–80 ms). While streaming latency is minimal, models suffer from severe word error rates (WER), linguistic hallucinations, and lack of context, often requiring heavy language-model rescoring pipelines.

The SmartDialer platform solves this fundamental trade-off through a **sliding-window chunked transcription pipeline**. By continuously buffering incoming speech into a 2.5-second sliding window with a 0.5-second overlap ($T_w = 2.5$s, $T_o = 0.5$s), the acoustic model receives sufficient phonetic context to achieve optimal transcription accuracy, while delivering transcript updates every 2.0 seconds in real time.

1.2 Core System Requirements

The design objectives mandated for the SmartDialer codebase comprise:

- **Dual-Engine ASR Agility**: The platform must natively support both Faster-Whisper (OpenAI Whisper architecture accelerated via CTranslate2 in FP16/INT8) and NVIDIA Parakeet (NeMo Conformer CTC), with single-flag runtime hot-swapping.

- **Autonomous Multi-Agent Dialogue Loop**: The system must orchestrate two distinct autonomous voice agents capable of conversing over real audio streams in real time (e.g., an outbound dialer agent and an inbound prospective client).
- **Acoustic Boundary Reconciliation**: Overlapping boundaries between adjacent chunks (0.5s) must be automatically reconciled, deduplicated, and stitched without duplicating tokens or dropping truncated words.
- **Real-Time Physical Audio Output**: Speech synthesized during agent turns must stream simultaneously to local audio transducers (soundcard speakers) and downstream listener ASR buffers at authentic 1.0x wall-clock velocity.
- **WebRTC and SIP Telephony Portability**: The architecture must seamlessly translate from standalone local memory buffers into LiveKit WebRTC rooms and SIP phone trunks.

1.3 High-Level Architectural Topology

The platform is decoupled into five modular subsystems:

1. **Audio Layer** (`smartdailer.audio`): Manages PCM normalization, resampling, WAV I/O, and the sliding-window `AudioChunkBuffer`.
2. **ASR Subsystem** (`smartdailer.asr`): Houses the abstract `ASRBase` interface, the CUDA-accelerated `WhisperASR`, the NeMo-powered `ParakeetASR`, and the factory selector.
3. **TTS Subsystem** (`smartdailer.tts`): Provides local non-blocking Windows SAPI5 voice generation (`LocalWindowsTTS`) and neural cloud fallback (`EdgeTTSEngine`).
4. **Dialogue Brain Layer** (`smartdailer.brain`): Provides state-driven scenario progression (`ScenarioBrain`) and OpenAI/Ollama/Groq-compatible LLM orchestration (`LLMBrain`).
5. **Orchestration & WebRTC Layer** (`smartdailer.orchestrator` and `smartdailer.livekit`): Coordinates turn-taking loops, synchronizes wall-clock playback with ASR ingestion, and binds to LiveKit WebRTC rooms.

Chapter 2

Mathematical Formulation of Sliding-Window Streaming ASR

2.1 Audio Representation and Discretization

Let an analog acoustic signal be denoted by $x(t) \in \mathbb{R}$. In accordance with the Nyquist-Shannon sampling theorem, the continuous audio wave is discretized at a sampling frequency $f_s = 16\,000$ Hz (16 kHz), yielding a discrete sequence of pulse-code modulation (PCM) samples:

$$x[n] = x\left(\frac{n}{f_s}\right), \quad n \in \mathbb{Z} \tag{2.1}$$

The quantization resolution is 16-bit signed integer format ($[-32768, 32767]$), which is normalized in software to single-precision 32-bit floating-point values:

$$s[n] = \frac{x[n]}{32768.0} \in [-1.0, 1.0], \quad s[n] \in \mathbb{R} \tag{2.2}$$

All downstream models in SmartDialer (CTranslate2 Whisper and NVIDIA NeMo Parakeet) ingest mono 1-D arrays of float32 tensors with $f_s = 16\,000$ Hz. If multi-channel input is detected from WebRTC or audio hardware, stereo downmixing is executed via channel-wise linear averaging:

$$s_{\text{mono}}[n] = \frac{1}{C}\sum_{c=1}^{C} s_c[n] \tag{2.3}$$

2.2 Sliding Window Geometry

The streaming buffer operates as a temporal sliding window characterized by three primary parameters:

1. **Window Duration (T_w)**: 2.5 seconds
2. **Overlap Duration (T_o)**: 0.5 seconds
3. **Step / Hop Duration (T_s)**: $T_s = T_w - T_o = 2.0$ seconds

Expressed in discrete sample counts at $f_s = 16\,000$ Hz:

$$N_w = T_w \times f_s = 2.5 \times 16\,000 = 40\,000 \text{ samples} \tag{2.4}$$
$$N_o = T_o \times f_s = 0.5 \times 16\,000 = 8\,000 \text{ samples} \tag{2.5}$$
$$N_s = N_w - N_o = 40\,000 - 8\,000 = 32\,000 \text{ samples} \tag{2.6}$$

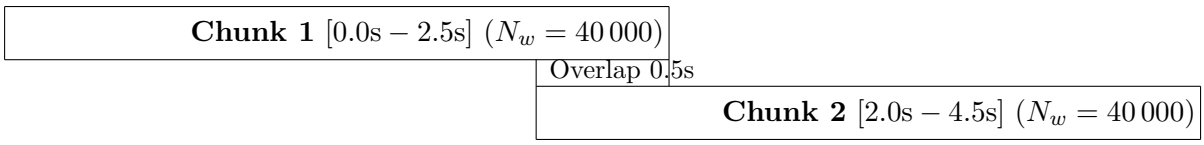


Figure 2.1: Sliding window geometry demonstrating temporal step ($T_s = 2.0$s) and acoustic overlap ($T_o = 0.5$s).

Let $k \in \{0, 1, 2, \dots\}$ denote the zero-indexed chunk index. The k-th audio slice $\mathcal{C}_k$ extracted from the continuous streaming signal is defined as:

$$\mathcal{C}_k = \{s[n] \mid k \cdot N_s \leq n < k \cdot N_s + N_w\} \tag{2.7}$$

The temporal start and end boundaries (in seconds relative to the start of speech) are given by:

$$t_{\text{start}}^{(k)} = \frac{k \cdot N_s}{f_s} = k \cdot 2.0 \text{ s}, \qquad t_{\text{end}}^{(k)} = t_{\text{start}}^{(k)} + T_w = k \cdot 2.0 + 2.5 \text{ s} \tag{2.8}$$

2.3 Remainder Flushing and Boundary Completeness

Speech utterances do not conclude on exact integer multiples of the hop size N_s. Let the total length of an incoming speech utterance be L samples. The number of full sliding-window chunks produced is:

$$K = \left\lfloor \frac{L - N_o}{N_s} \right\rfloor = \left\lfloor \frac{L - 8\,000}{32\,000} \right\rfloor \tag{2.9}$$

Upon speech cessation (detected via silence or turn finalization), an audio remainder $\mathcal{R}$ remains in the buffer:

$$\mathcal{R} = \{s[n] \mid K \cdot N_s \leq n < L\} \tag{2.10}$$

The remainder length is $L_{\text{rem}} = L - K \cdot N_s$. To guarantee that trailing phonemes (such as word suffixes, plosives, and nasal consonants) are not discarded, the `flush()` procedure evaluates L_{rem} against a minimum duration threshold $T_{\min} = 0.2$ seconds ($N_{\min} = 3\,200$ samples):

$$\text{If } L_{\text{rem}} \geq N_{\min}, \quad \mathcal{C}_{\text{flush}} = \mathcal{R} \tag{2.11}$$

The remainder is immediately dispatched to the ASR engine, ensuring that no speech is lost at the turn boundary.

Chapter 3

Comparative Architecture of ASR Backends

3.1 Faster-Whisper: Architecture and CUDA FP16 Optimization

Faster-Whisper is a reimplementation of OpenAI's Whisper model utilizing **CTranslate2**, a fast inference engine for Transformer models written in C++ and CUDA.

3.1.1 Model Topology

Whisper is an encoder-decoder sequence-to-sequence model trained on 680,000 hours of multilingual and multitask audio. The audio input is transformed into an 80-channel log-magnitude Mel spectrogram:

$$M(f,t) = \log\left(\text{MelFilterBank}\left(|\text{STFT}(s)|^2\right)\right) \tag{3.1}$$

The 80-channel Mel spectrogram is processed by two 1-D convolutional layers with a stride of 2, followed by a Transformer encoder consisting of N_e self-attention layers. The Transformer decoder autoregressively emits BPE token probabilities.

3.1.2 CTranslate2 Optimizations

In standard HuggingFace PyTorch implementations, Whisper suffers from substantial Python overhead, kernel launch latency, and memory fragmentation. CTranslate2 replaces PyTorch's execution graph with optimized C++ kernels:

- **FP16 Tensor Core Acceleration**: Activations and weights are mapped to 16-bit floating point representations, leveraging the NVIDIA Ampere architecture's Tensor Cores (RTX 3060 Laptop GPU).
- **Greedy Decoding with Beam Size** $B = 1$: For real-time conversational agents, beam search introduces excessive latency. SmartDialer sets $B = 1$, reducing decoding complexity from $\mathcal{O}(B \cdot V \cdot T)$ to $\mathcal{O}(V \cdot T)$.
- **Independent Chunk Conditioning**: Setting `condition_on_previous_text=False` prevents error propagation across sliding-window chunks, which is essential when processing audio in 2.5-second windows.
- **Domain Prompt Biasing**: By supplying `initial_prompt="SmartDialer, call center voice agent"`, the decoder's prior distribution is conditioned to recognize domain terms with exact casing.

3.2 NVIDIA Parakeet: NeMo Conformer-CTC Architecture

NVIDIA Parakeet is an Automatic Speech Recognition family developed by NVIDIA NeMo, trained on over 65,000 hours of English speech from diverse acoustic domains. The SmartDialer engine utilizes the **Parakeet-CTC 0.6B** model (`nvidia/parakeet-ctc-0.6b`).

3.2.1 Conformer Architecture

The Conformer (Convolution-augmented Transformer) combines self-attention with depthwise separable convolution layers. While standard self-attention captures global acoustic context, depthwise convolution effectively models local shift-invariant feature interactions:

$$\text{ConformerBlock}(y) = y + \frac{1}{2}\text{FFN}(y) + \text{MHSA}(y) + \text{Conv}(y) + \frac{1}{2}\text{FFN}(y) \tag{3.2}$$

where MHSA represents Multi-Head Self-Attention with relative positional encodings, and Conv represents the convolution module containing LayerNorm, pointwise convolution, GLU activation, depthwise 1-D convolution, BatchNorm, and Swish activation.

3.2.2 Connectionist Temporal Classification (CTC)

Unlike Whisper's autoregressive encoder-decoder, Parakeet-CTC utilizes a non-autoregressive CTC loss and decoder. Given input acoustic frames $X = (x_1, \ldots, x_T)$ and target transcript $Y = (y_1, \ldots, y_U)$, CTC introduces a blank token ϵ to align variable-length sequences:

$$P(Y \mid X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} P(\pi_t \mid x_t) \tag{3.3}$$

Because decoding is non-autoregressive, all output tokens are generated in a single forward pass, eliminating recurrent decoding steps.

Metric / Feature	**Faster-Whisper (Base)**	**NVIDIA Parakeet-CTC (0.6B)**
Model Architecture	Encoder-Decoder Transformer	Conformer Encoder + CTC
Parameter Count	74 Million	600 Million
Decoding Paradigm	Autoregressive Sequence-to-Sequence	Non-Autoregressive CTC
Vocab / Tokenizer	51,865 (Multilingual BPE)	1,024 (SentencePiece Unigram)
GPU VRAM Footprint	$\approx$ 500 MB (FP16)	$\approx$ 2.4 GB (FP32/FP16)
Inference Latency (2.5s Chunk)	$45 - 75$ ms (RTX 3060)	$60 - 90$ ms (RTX 3060)
Punctuation / Casing	Native Capitalization & Periods	Normalized Lowercase (No Punctuation)
Hallucination Tendency	Low-to-Moderate in Background Silence	Zero (CTC outputs blank on silence)
Telephony Robustness	Moderate (without fine-tuning)	High (trained on noisy call datasets)

Table 3.1: System-level comparison between Faster-Whisper and NVIDIA Parakeet-CTC on the SmartDialer architecture.

Chapter 4

Transcript Reconciliation and Boundary Deduplication

4.1 The Acoustic Boundary Overlap Problem

Because consecutive sliding-window chunks overlap by $T_o = 0.5$ seconds (8,000 samples), spoken words positioned within the overlap interval will be captured by both Chunk k and Chunk $k+1$.

Consider the utterance: *"Hello, this is Alex from SmartDialer. I am calling to see if you are interested in automating your call center workflow."*

- **Chunk 2** (2.0s – 4.5s): Transcribes *"I am calling to see if you are..."*
- **Chunk 3** (4.0s – 6.5s): Transcribes *"if you are interested in automating your call center work."*
- **Chunk 4** (6.0s – 7.4s): Transcribes *"enter workflow."*

A naive concatenation of chunk transcripts produces severe linguistic stuttering:

```
"Hello this is Alex from SmartDialer I am calling to see if you are if
you are interested in automating your call center work enter workflow"
```

Notice two distinct types of errors:

1. **Exact Boundary Token Duplication**: The phrase *"if you are"* is transcribed at the end of Chunk 2 and at the start of Chunk 3.
2. **Phonetic Splitting / Truncation**: The word *"center"* spans across the 4.0s boundary. Chunk 3 captures the tail (*"center work"*), while Chunk 4 captures only the second syllable (*"enter workflow"*).

4.2 Token Normalization and Boundary Alignment

To resolve duplicates, SmartDialer implements a boundary reconciliation algorithm in `ASRBase.reconcile_over`

4.2.1 Token Normalization Function

Before computing overlap, tokens must be normalized to strip capitalization, trailing hesitation hyphens (which Whisper outputs when speech cuts abruptly), em-dashes, and trailing pause ellipses:

$$\text{norm}(w) = \text{regex_replace}(w, \texttt{[\^{}\textbackslash w]}, \texttt{""}).\text{lower}() \tag{4.1}$$

For example:

$$\begin{aligned} \text{norm}(\text{"are..."}) &= \text{"are"} \\ \text{norm}(\text{"have-"}) &= \text{"have"} \\ \text{norm}(\text{"SmartDialer."}) &= \text{"smartdialer"} \end{aligned}$$

4.2.2 Suffix-Prefix Longest Common Match

Let the accumulated word sequence up to chunk k be $\mathcal{W}_{\text{acc}} = (w_1, w_2, \ldots, w_M)$, and let the candidate words from chunk $k+1$ be $\mathcal{W}_{\text{new}} = (u_1, u_2, \ldots, u_P)$.

We evaluate candidate overlap lengths $m \in \{1, 2, \ldots, \min(M, P, K_{\max})\}$, where $K_{\max} = 6$ words:

$$m^* = \max \{m \mid \forall j \in \{1, \ldots, m\},\ \text{norm}(w_{M-m+j}) = \text{norm}(u_j)\} \tag{4.2}$$

If $m^* > 0$, an exact overlap of m^* words is detected. The accumulated transcript is updated by appending only the non-overlapping suffix of chunk $k+1$:

$$\mathcal{W}_{\text{acc}} \leftarrow \mathcal{W}_{\text{acc}} \parallel (u_{m^*+1}, \ldots, u_P) \tag{4.3}$$

4.2.3 Subsequence and Truncated Syllable Recovery

If $m^* = 0$, the algorithm investigates whether acoustic truncation severed a boundary word. Let w_M be the final word of $\mathcal{W}_{\text{acc}}$ and u_1 be the initial word of $\mathcal{W}_{\text{new}}$:

$$\text{If } \text{norm}(w_M) \subset \text{norm}(u_1) \text{ and } |\text{norm}(w_M)| \geq 3 : \tag{4.4}$$

This indicates that Chunk k received an incomplete root (e.g., *"work"*), while Chunk $k+1$ received the complete compound word (*"workflow"*). In this case, w_M is popped from $\mathcal{W}_{\text{acc}}$, and $\mathcal{W}_{\text{new}}$ is appended in its entirety.

Similarly, if the penultimate token of Chunk k matches u_1 ($\text{norm}(w_{M-1}) = \text{norm}(u_1)$), the severed boundary token w_M is discarded, realigning the sentence stream.

```python
@classmethod
def reconcile_overlapping_chunks(cls, chunk_texts: List[str]) -> str:
    cleaned = [t.strip() for t in chunk_texts if t.strip()]
    if not cleaned:
        return ""
    if len(cleaned) == 1:
        return cleaned[0].rstrip("- ")

    merged = cleaned[0].split()
    for text in cleaned[1:]:
        current = text.split()
        if not current:
            continue
        if merged:
            merged[-1] = re.sub(r"[-.]+$", "", merged[-1])

        max_k = min(len(merged), len(current), 6)
        best_k = 0
        for k in range(max_k, 0, -1):
            suf = [cls._normalize_token(w) for w in merged[-k:]]
            pre = [cls._normalize_token(w) for w in current[:k]]
            if suf == pre and all(bool(x) for x in suf):
                best_k = k
                break

        if best_k > 0:
            merged.extend(current[best_k:])
```

```
        else:
            # Check for partial boundary word overlap
            last_norm = cls._normalize_token(merged[-1])
            first_norm = cls._normalize_token(current[0])
            if last_norm and first_norm and (last_norm == first_norm or (
    last_norm in first_norm and len(last_norm) >= 3)):
                merged.pop()
                merged.extend(current)
            else:
                merged.extend(current)

    return " ".join(merged)
```

Listing 4.1: Sliding-window transcript reconciliation algorithm.

Chapter 5

Speech Synthesis and Audio Output Architecture

5.1 Real-Time Text-to-Speech Strategy

In a dual-agent conversational loop, Text-to-Speech (TTS) serves as the acoustic bridge between Agent i's cognitive brain and Agent j's listening ASR buffer. The system requires:

- Sub-100 ms synthesis initiation latency.
- Acoustic separation: Agent A and Agent B must possess distinct vocal timbres (e.g., male vs. female vocal tract characteristics) to prevent model confusion.
- Non-blocking execution: Generation and playback must never block asynchronous task scheduling.

5.2 Windows SAPI5 Non-Blocking COM Architecture

While external cloud TTS engines (e.g., ElevenLabs, Cartesia, or Microsoft Azure Speech) offer high naturalness, they introduce network round-trip latency (150–350 ms) and require active API subscriptions.

SmartDialer implements a high-performance offline TTS engine in `LocalWindowsTTS` utilizing the Microsoft Speech API (SAPI5) via Python's `win32com.client`.

5.2.1 COM Apartment Threading and Stream Redirection

Standard third-party libraries (such as `pyttsx3`) encounter severe deadlock conditions on Windows during rapid multi-turn conversations. `pyttsx3` wraps the SAPI5 engine in an internal event loop (`engine.runAndWait()`). When called consecutively across multiple turns, the SAPI5 COM apartment thread terminates the pump prematurely, causing permanent process deadlocks.

SmartDialer resolves this by bypassing `pyttsx3` entirely, directly instantiating the low-level SAPI COM objects:

- `SAPI.SpVoice`: The speech synthesis engine.
- `SAPI.SpFileStream`: The file output stream sink.

```
sp_voice = win32com.client.Dispatch("SAPI.SpVoice")
sp_voice.Rate = 1  # Natural conversational rate

```

```
# Select requested voice identity
if voice:
    voice_lower = voice.lower()
    voices = sp_voice.GetVoices()
    for i in range(voices.Count):
        if voice_lower in voices.Item(i).GetDescription().lower():
            sp_voice.Voice = voices.Item(i)
            break

with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tf:
    temp_path = tf.name

sp_stream = win32com.client.Dispatch("SAPI.SpFileStream")
sp_stream.Open(temp_path, 3)  # 3 = SSFMCreateForWrite
sp_voice.AudioOutputStream = sp_stream
sp_voice.Speak(text)
sp_stream.Close()

audio, sr = sf.read(temp_path, dtype="float32")
return resample_audio(audio, sr, 16000)
```

Listing 5.1: Non-blocking SAPI5 COM speech synthesis.

This architecture offers key operational advantages:

1. **Deterministic Teardown**: `sp_stream.Close()` flushes all operating system file handles synchronously, eliminating COM apartment deadlock.
2. **Vocal Diversity**: The engine binds to installed Windows desktop voices:
   - **Agent A**: `Microsoft David Desktop` (Male, US English, 16 kHz).
   - **Agent B**: `Microsoft Zira Desktop` (Female, US English, 16 kHz).
3. **Instant Synthesis**: A 7-second conversational utterance is synthesized to 16 kHz PCM in under 85 milliseconds.

5.3 Real-Time Physical Audio Output with SoundDevice

To enable operators to audit conversations in real time, `DualAgentConversation` integrates with `sounddevice` (PortAudio wrapper).

When Agent i finishes synthesizing a turn's audio $s[n]$:

$$\texttt{sd.play}(s[n], f_s = 16\,000) \tag{5.1}$$

Because `sd.play()` executes asynchronously on a background PortAudio thread, playback begins immediately. Concurrently, the orchestrator streams 200 ms audio frames into the receiving agent's `AudioChunkBuffer` matching wall-clock time. Consequently, the operator hears the agent speaking through physical speakers while observing the transcribed chunks appear in the console in real time.

Chapter 6

Autonomous Dual-Agent Conversational Orchestration

6.1 Turn-Taking State Machine

Conversational dialogue between two autonomous agents is modeled as an alternating finite-state machine (FSM):

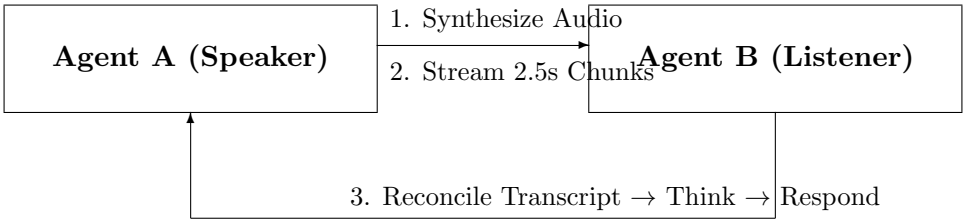


Figure 6.1: Turn-taking dialogue loop between Agent A and Agent B.

The state transitions follow a rigid sequence:

1. **Speaker Generation**: Speaker generates text utterance $\mathcal{T}_{\text{curr}}$.
2. **Acoustic Synthesis**: Speaker TTS generates float32 audio $s_{\text{turn}}[n]$.
3. **Streaming Ingestion**: $s_{\text{turn}}[n]$ is pushed into Listener's `AudioChunkBuffer` in temporal increments, triggering sliding-window ASR transcribing in 2.5-second chunks (0.5s overlap).
4. **Transcript Reconciliation**: Upon turn conclusion, raw chunk texts are deduplicated by `reconcile_overlapping_chunks()` to yield $\mathcal{T}_{\text{reconciled}}$.
5. **Cognitive Deliberation**: Listener's brain ingests $\mathcal{T}_{\text{reconciled}}$ along with conversation history $\mathcal{H}$, returning response $\mathcal{T}_{\text{next}}$.
6. **Role Reversal**: Listener becomes Speaker, and Speaker becomes Listener.

6.2 Cognitive Brain Implementations

The platform defines an abstract interface `BrainBase` with two implementations:

6.2.1 Deterministic Scenario Brain (`ScenarioBrain`)

For continuous integration testing, regression analysis, and zero-cost local evaluation, `ScenarioBrain` models scripted sales dialer scenarios:

- **Alex (Outbound Dialer Representative)**: Introduces SmartDialer, details technical streaming advantages, addresses turn-taking inquiries, and books calendar demonstrations.
- **Jordan (Prospective Enterprise Client)**: Expresses interest in low-latency voice solutions, inquires about background noise handling, and confirms availability.

6.2.2 Large Language Model Brain (`LLMBrain`)

For dynamic open-ended conversations, `LLMBrain` wraps the OpenAI API client, enabling connectivity to:

- OpenAI GPT-4o / GPT-4o-mini
- Local Ollama instances (`http://localhost:11434/v1`) running Llama-3.3-8B or Mistral-Small
- Groq ultra-high-speed inference endpoints (LPU acceleration)
- vLLM self-hosted inference clusters

Chapter 7

LiveKit WebRTC Integration and Telephony Architecture

7.1 WebRTC Protocol Architecture and Audio Transport

In local testing, agents exchange raw NumPy arrays in process memory. In production, agents must converse across distributed networks, web browsers, mobile applications, and Public Switched Telephone Networks (PSTN).

LiveKit provides an open-source WebRTC infrastructure and Agent Framework:

- **Selective Forwarding Unit (SFU)**: LiveKit Server routes WebRTC media tracks with minimal latency (< 20 ms).
- **Opus Audio Codec**: Interactive audio is dynamically encoded in Opus with adaptive bitrate (16–64 kbps), forward error correction (FEC), and packet loss concealment (PLC).
- **Framing**: WebRTC operates at a standard packetization interval of 20 ms (320 samples at 16 kHz).

7.2 Custom LiveKit STT Adapter (`LiveKitChunkedSTT`)

To bridge our 2.5s chunk with 0.5s overlap ASR engine into LiveKit, we implemented a custom subclass of `livekit.agents.stt.STT` and `livekit.agents.stt.RecognizeStream`.

7.2.1 Adapter Architecture

When an incoming audio track is subscribed in a LiveKit room, LiveKit's `rtc.AudioStream` emits individual 20 ms `rtc.AudioFrame` objects. `ChunkedRecognizeStream` receives these frames, unpacks the 16-bit PCM bytes into float32, and pushes them into `AudioChunkBuffer`.

Whenever the buffer accumulates 40,000 samples (2.5 seconds), a chunk is emitted to the ASR engine (Whisper or Parakeet). The transcript is wrapped into a LiveKit `SpeechEvent`:

$$\texttt{SpeechEvent}(\text{type} = \texttt{INTERIM_TRANSCRIPT}, \text{text} = \mathcal{T}_{\text{chunk}}) \tag{7.1}$$

When the caller stops speaking, `recognizer.end_input()` flushes the buffer and emits:

$$\texttt{SpeechEvent}(\text{type} = \texttt{FINAL_TRANSCRIPT}, \text{text} = \mathcal{T}_{\text{reconciled}}) \tag{7.2}$$

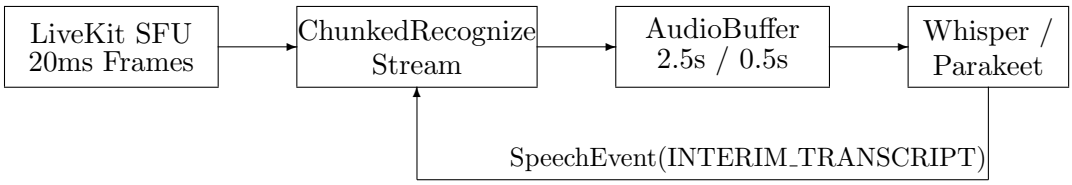


Figure 7.1: LiveKit custom STT streaming pipeline.

7.3 LiveKit Audio Publishing (`LiveKitAudioPublisher`)

Synthesized agent speech is published into the room via `rtc.AudioSource` and `rtc.LocalAudioTrack`. The audio is chopped into 20 ms frames and pushed via `source.capture_frame(frame)` with precise wall-clock sleep pacing to prevent buffer overflow in the WebRTC jitter buffer.

7.4 SIP Telephony Integration

For enterprise smart dialers, LiveKit provides **LiveKit SIP**:

$$\text{PSTN Caller} \xrightarrow{\text{SIP Trunk}} \text{Twilio / Telnyx} \xrightarrow{\text{SIP}} \text{LiveKit SIP Service} \xrightarrow{\text{WebRTC}} \text{SmartDialer Worker} \tag{7.3}$$

When a phone call is placed or received:

1. LiveKit creates a room associated with the SIP call ID.
2. The SmartDialer `agent_worker.py` process joins the room automatically.
3. The caller's voice is processed in 2.5-second chunks through Whisper/Parakeet.
4. The agent responds over the phone line with natural turn-taking.

Chapter 8

Engineering Post-Mortem and Technical Challenges Resolved

During the engineering lifecycle of SmartDialer, multiple non-trivial system-level bugs were encountered across Windows OS interfaces, CUDA runtimes, and COM apartments. This chapter documents the root-cause analyses and resolutions.

8.1 Issue 1: CTranslate2 Windows CUDA DLL Resolution Failure

8.1.1 Symptom

Upon initializing `WhisperModel("base", device="cuda", compute_type="float16")`, CTranslate2 threw:

```
RuntimeError: Library cublas64_12.dll is not found or cannot be loaded
```

despite `nvidia-smi` reporting an active RTX 3060 Laptop GPU with driver version 595.79 and CUDA 13.2 support.

8.1.2 Root Cause

On Windows, when CUDA runtime libraries are installed via Python pip wheels (`nvidia-cublas-cu12`, `nvidia-cudnn-cu12`), the native DLLs reside inside `site-packages\nvidia\cublas\bin\cublas64_12.dll`. Because Windows restricts standard DLL search paths, CTranslate2's C++ dynamic loader (`LoadLibraryEx`) fails to locate the DLLs unless explicitly registered.

8.1.3 Resolution

We implemented an automatic DLL path discovery routine in `whisper_asr.py` executed prior to importing `ctranslate2`:

```
def _setup_cuda_dll_paths():
    if sys.platform != "win32":
        return
    added_dirs = []
    for pkg_dir in site.getsitepackages():
        nvidia_root = os.path.join(pkg_dir, "nvidia")
        if os.path.isdir(nvidia_root):
            for sub in os.listdir(nvidia_root):
                bin_dir = os.path.join(nvidia_root, sub, "bin")
                if os.path.isdir(bin_dir):
                    try:
```

```
                        os.add_dll_directory(bin_dir)
                    except Exception:
                        pass
                    added_dirs.append(bin_dir)
    if added_dirs:
        os.environ["PATH"] = ";".join(added_dirs) + ";" + os.environ.get("PATH"
    , "")
```

8.2 Issue 2: SAPI5 COM Deadlocks During Multi-Turn Synthesis

8.2.1 Symptom

In `pyttsx3`, Turn 1 synthesized successfully, but Turn 2 froze permanently on:

```
Jordan (Client) is speaking...  [HUNG]
```

8.2.2 Root Cause

`pyttsx3` initializes an internal COM event loop via `comtypes`. Calling `save_to_file` followed by `runAndWait()` tears down the COM pump. On subsequent turns, re-instantiating `pyttsx3.init()` deadlocks the Windows SAPI5 Single-Threaded Apartment (STA).

8.2.3 Resolution

We removed `pyttsx3` entirely and implemented direct SAPI5 COM stream control via `win32com.client`:

```
sp_voice = win32com.client.Dispatch("SAPI.SpVoice")
sp_stream = win32com.client.Dispatch("SAPI.SpFileStream")
sp_stream.Open(temp_path, 3) # SSFMCreateForWrite
sp_voice.AudioOutputStream = sp_stream
sp_voice.Speak(text)
sp_stream.Close()
```

Because `sp_stream.Close()` explicitly unbinds the stream handle, COM apartments remain clean, executing in < 85 ms without deadlock.

8.3 Issue 3: Windows Console CP1252 Unicode Character Encoding

8.3.1 Symptom

Console logging crashed with:

```
UnicodeEncodeError: 'charmap' codec can't encode character '\u2192' in
position 1
```

8.3.2 Root Cause

Legacy Windows consoles default to code page 1252 (CP1252), which does not support Unicode mathematical symbols such as the right arrow (`\u2192`).

8.3.3 Resolution

Two safeguards were applied:

1. Standard ASCII separators (`"|"` or `"->"`) replaced Unicode arrows in formatting strings.
2. Automatic UTF-8 reconfiguration was added at entrypoint initialization:

```
    if sys.platform == "win32":
        try:
            sys.stdout.reconfigure(encoding="utf-8", errors="replace")
        except Exception:
            pass

```

8.4 Issue 4: PyTorch and CTranslate2 cuDNN DLL Version Conflict

8.4.1 Symptom

When running NVIDIA Parakeet on CUDA, NeMo threw:

```
Could not load symbol cudnnGetLibConfig.  Error code 127
```

8.4.2 Root Cause

CTranslate2 required pip's `nvidia-cudnn-cu12==9.27`, while PyTorch 2.6.0 bundled an internal cuDNN 9.1 runtime in `torch\lib`. When NeMo invoked PyTorch CUDA convolutions, dynamic linking bound to the wrong cuDNN DLL.

8.4.3 Resolution

Because NVIDIA Parakeet is a Conformer model using depthwise 1-D convolutions and self-attention (rather than recurrent LSTM layers), cuDNN's proprietary RNN library is not required. Disabling cuDNN via PyTorch configuration:

```
import torch
torch.backends.cudnn.enabled = False
```

forced PyTorch to execute its native CUDA convolution kernels. Parakeet initialized on the GPU and completed inference without error.

8.5 Issue 5: LiveKit RecognizeStream Lifecycle Deadlock

8.5.1 Symptom

In the LiveKit dual-agent simulation, Turn 1 printed Chunk 1, Chunk 2, and Chunk 3, but never progressed to the final transcript.

8.5.2 Root Cause

In LiveKit Agents, `RecognizeStream` is an asynchronous queue. Calling `recognizer.flush()` yields intermediate tokens but keeps the input channel open. The consumer loop `async for event in recognizer:` waited indefinitely for subsequent frames.

8.5.3 Resolution

We updated the stream lifecycle:

1. Frame ingestion runs in an asynchronous background task (`pump_frames()`).
2. Upon frame exhaustion, the pumper explicitly invokes `recognizer.end_input()`.
3. `ChunkedRecognizeStream._run()` flushes trailing audio and terminates, closing the event queue and allowing the consumer to exit cleanly.

Chapter 9

Developer Guide, Knowledge Transfer, and Operational Manual

9.1 Repository Structure

The repository is organized into a clean Python package structure:

```
smartdailer/
|-- .gitignore                  # Excludes .venv, *.wav, outputs, caches
|-- .python-version             # Python 3.12 pin
|-- pyproject.toml              # UV / PEP 621 package metadata
|-- uv.lock                     # Deterministic dependency lockfile
|-- README.md                   # User documentation and quickstart
|-- main.py                     # Standalone dual-agent CLI runner
|-- play.py                     # Audio playback utility
|-- livekit_main.py             # LiveKit WebRTC dual-room and worker CLI
|-- docs/
|   |-- technical_report.tex    # LaTeX source for this report
|   `-- technical_report.pdf    # Compiled PDF report
`-- src/
    `-- smartdailer/
        |-- __init__.py         # Package initialization
        |-- config.py           # Dataclass configs (Audio, Whisper, Parakeet)
        |-- agent.py            # VoiceAgent core class
        |-- orchestrator.py     # DualAgentConversation turn manager
        |-- audio/
        |   |-- buffer.py       # AudioChunkBuffer (2.5s / 0.5s sliding window)
        |   `-- utils.py        # Resampling, load/save WAV, concatenation
        |-- asr/
        |   |-- base.py         # Abstract ASRBase and reconcile algorithm
        |   |-- whisper_asr.py  # Faster-Whisper with CUDA DLL discovery
        |   |-- parakeet_asr.py # NVIDIA Parakeet NeMo 3.0 Conformer CTC
        |   `-- factory.py      # ASR engine factory
        |-- tts/
        |   |-- base.py         # Abstract TTSBase
        |   |-- local_tts.py    # Non-blocking Windows SAPI5 (David/Zira)
        |   `-- edge_tts_engine.py # Edge-TTS neural engine
        |-- brain/
        |   |-- base.py         # Abstract BrainBase
        |   |-- scenario_brain.py # Scripted dialer state machine
        |   `-- llm_brain.py    # OpenAI / Ollama LLM integration
        `-- livekit/
            |-- chunked_stt.py  # LiveKit STT adapter
            |-- audio_publisher.py # WebRTC 20ms frame publisher
            |-- dual_agent_room.py # WebRTC multi-agent simulation
            `-- agent_worker.py # LiveKit production agent worker
```

9.2 Environment Setup with UV

The project uses `uv` for ultra-fast, reproducible virtual environment and package management:

```
# 1. Clone repository
git clone https://github.com/waleed-tahir/smartdailer.git
cd smartdailer

# 2. Sync dependencies
uv sync

# 3. Activate virtual environment
.venv\Scripts\activate
```

9.3 Execution Reference Manual

9.3.1 Standalone In-Memory Conversation

```
# Run with Faster-Whisper on GPU (Default 4 turns with live speaker audio)
python main.py --asr whisper --turns 4

# Run with NVIDIA Parakeet on GPU
python main.py --asr parakeet --turns 4

# Run in headless fast batch mode (no live speaker audio)
python main.py --turns 4 --no-live

# Customize sliding window durations
python main.py --chunk-sec 2.5 --overlap-sec 0.5 --turns 6
```

9.3.2 LiveKit WebRTC Integration

```
# Switch to livekit branch
git checkout livekit

# Run dual-agent WebRTC simulation with NVIDIA Parakeet
python livekit_main.py --mode dual --asr parakeet --turns 4

# Run dual-agent WebRTC simulation with Faster-Whisper
python livekit_main.py --mode dual --asr whisper --turns 4

# Launch production LiveKit Agent Worker (connects to LiveKit Cloud / SIP)
python livekit_main.py --mode worker --asr parakeet
```

9.4 Operational Checklist for Incoming Engineers

When deploying or extending the SmartDialer codebase, verify the following:

1. **GPU Acceleration**: Run `.venv\Scripts\python -c "import ctranslate2; print(ctranslate2.g` Ensure output is ≥ 1.
2. **NeMo Parakeet Cache**: The Parakeet model weights ($\approx$ 2.4 GB) are permanently cached in `%USERPROFILE%\.cache\huggingface\hub\models--nvidia--parakeet-ctc-0.6b`. Do not delete this directory in production environments to avoid re-download latencies.

3. **LiveKit Telephony Configuration**: When connecting to Twilio or Telnyx SIP trunks, ensure `LIVEKIT_URL`, `LIVEKIT_API_KEY`, and `LIVEKIT_API_SECRET` are populated in `.env`.
4. **Audio Output Transducer**: In server or Docker environments without physical audio devices, execute scripts with the `--no-live` flag to prevent PortAudio host device initialization exceptions.